Ceph-SRE: An Autonomous LLM Agent for On-Node Ceph Cluster Management

Vladislav Shabanov
University of California, Santa Cruz
Santa Cruz, California, USA
vshabano@ucsc.edu

Abstract

Managing Ceph storage clusters remains largely manual: administrators chain CLI commands, interpret JSON output, and follow ad-hoc runbooks to diagnose failures. We present Ceph-SRE, an autonomous agent that runs *locally* on commodity storage nodes and replaces brittle operator scripts with a *hybrid control loop*. A *Fast Path* handles routine queries ("list pools", "OSD status") through deterministic intent classification in a single LLM call, while an *Agent Path* activates a multi-step *ReAct* reasoning loop for complex diagnostics ("fix degraded PGs"). Both paths are protected by a four-tier safety framework that classifies every action by risk level and gates destructive operations on explicit confirmation. We evaluate Ceph-SRE on an NVIDIA P100 node with five quantised LLMs (3.8 B–14 B parameters) across 161 test cases spanning intent classification, autonomous reasoning, safety enforcement, and anomaly detection. A 7 B model achieves 85.5% intent accuracy with 6 s mean latency, while safety classification and anomaly detection reach 100% accuracy regardless of model size—validating our design choice to decouple deterministic safety logic from probabilistic LLM inference.

1 Introduction

Ceph [1] is the dominant open-source distributed storage system, underpinning block, file, and object workloads in production clouds. Yet day-to-day operations—diagnosing degraded placement groups, rebalancing OSDs after a disk replacement, predicting capacity exhaustion—still depend on administrators manually invoking `ceph osd tree`, `ceph pg dump`, and similar commands, then mentally correlating the output. Site-reliability teams encode this knowledge in shell scripts that are brittle, cluster-specific, and difficult to maintain.

Recent advances in small language models (SLMs) make it feasible to run 7–14 B-parameter models on the same commodity hardware that hosts Ceph OSDs. This raises a compelling systems question: *can an LLM-based agent replace fragile operator scripts while running entirely on-node, within the latency and safety envelope required for storage management?*

We answer this with Ceph-SRE, a hybrid agent architecture whose key contributions are:

- A **dual-mode control loop** that routes simple CLI tasks to a deterministic fast path (single LLM call) and complex diagnostics to a multi-step ReAct [2] reasoning loop, achieving 100% routing accuracy across all tested models.
- A **four-tier safety framework** that classifies 34 Ceph operations into low/medium/high/critical risk levels, reaching 100% risk classification accuracy independent of the underlying LLM.
- A **threshold-based anomaly detector** that scores cluster health across seven categories (health, OSD, PG, capacity, balance, performance, configuration) with perfect F1 on synthetic fault scenarios.
- An **empirical evaluation** on an NVIDIA P100 (12 GB) comparing five local LLMs across 161 test cases, showing that a 7 B quantised model is the practical sweet spot for on-node deployment.

2 Design

Figure **??** shows the overall architecture. Ceph-SRE is structured as four cooperating subsystems: a *Hybrid Router* (§2.1), a *ReAct Agent Loop* (§2.2), a *Safety Framework* (§2.3), and an *Anomaly Detector* (§2.4). All components share a *Tool Registry* of 35 Ceph operations and five pre-authored runbooks.

2.1 Hybrid Router

On each user query, the router decides between two execution paths:

Fast Path (Simple Mode). If the query contains none of 24 agent-trigger keywords (e.g., "troubleshoot", "diagnose", "fix", "rebalance"), the router issues a single LLM function-call to classify the *intent* and extract *parameters*, then dispatches to the corresponding tool. This path completes in one LLM round-trip.

Agent Path (ReAct Mode). Queries containing agent keywords are forwarded to the autonomous reasoning loop (§2.2). The keyword check is a zero-cost string match that avoids an extra LLM call for mode selection, contributing to the 100% routing accuracy we observe (§3).

2.2 ReAct Agent Loop

Complex tasks are handled by a ReAct loop that iterates up to 10 steps (configurable) within a 120 s session budget:

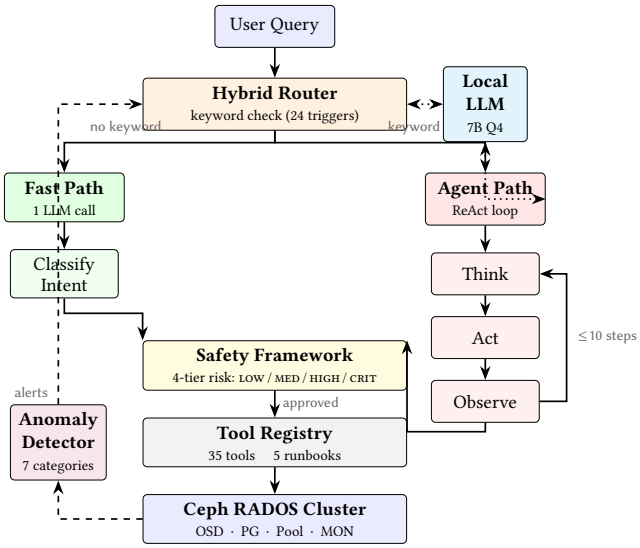

Figure 1. Ceph-SRE architecture. The Hybrid Router directs queries to a single-call Fast Path or a multi-step ReAct Agent Path. Both paths pass through the Safety Framework before reaching the Tool Registry. The Anomaly Detector monitors cluster state independently and feeds alerts back to the router. The local LLM (dashed) serves both intent classification and autonomous reasoning.

1. **Think.** The LLM receives the full tool schema, the current scratchpad of prior observations, and conversation context. It emits a structured response containing a *Thought*, an *Action* (tool name), and *Action Input* (parameters); or a *Final Answer* if reasoning is complete.
2. **Act.** If the selected action is in the *destructive set* (`delete_pool`, `remove_osd`, etc.), the safety framework (§2.3) must approve it before execution. The tool's return value is recorded as an *Observation*.
3. **Loop.** Steps repeat until a final answer is produced, the iteration limit is reached (in which case the scratchpad is summarised), or the time budget expires.

The loop has access to 35 tools across seven categories: object storage (11), cluster monitoring (8), cluster management (12), documentation/RAG (2), configuration (2), runbook execution (3), and planning (2). Five pre-authored runbooks encode multi-step remediation procedures (e.g., *Recover Down OSD*: health check → OSD status → PG impact → restart → verify).

2.3 Safety Framework

Every action—whether initiated by the fast path or the agent loop—passes through the `ActionEngine`, which enforces a four-tier risk model over 34 mapped operations:

Risk Level	Actions	Policy
LOW	20	Auto-approve (read-only)
MEDIUM	8	Log, auto-approve
HIGH	8	Require confirmation
CRITICAL	4	Require confirmation + log

An `ActionPolicy` object further constrains execution via session-level rate limits (default 20 actions), dry-run mode, and per-action allow/block lists. Rollback templates exist for six reversible operations (e.g., `set_osd_out` ↔ `set_osd_in`). Crucially, risk classification is *deterministic*—it is a dictionary lookup, not an LLM inference—so its correctness is independent of model quality.

2.4 Anomaly Detector

The anomaly detector periodically scores cluster state across seven categories using configurable thresholds (e.g., OSD utilisation warning at 75%, critical at 85%; PG degraded > 0 triggers a warning). Each detected anomaly carries a severity (INFO/WARNING/CRITICAL), and the overall cluster score is computed as:

$$S = \max\Big(0,\ 100 - \sum_{\text{anomalies}} p_i\Big), \quad p_i = \begin{cases} 25 & \text{critical} \\ 10 & \text{warning} \\ 2 & \text{info} \end{cases}$$

Anomalies are linked to suggested runbooks for automated remediation. Like the safety framework, this component is deterministic: it does not invoke the LLM.

3 Evaluation

We evaluate Ceph-SRE across four axes: intent classification accuracy (§3.1), ReAct reasoning quality (§3.2), safety enforcement (§3.3), and anomaly detection (§3.3). Latency is reported throughout.

Setup. All experiments run on a single CloudLab node with an NVIDIA P100 GPU (12 GB VRAM), representing commodity hardware co-located with a Ceph OSD. We serve models via Ollama and test five configurations: `phi3.5` (3.8 B, Q4), `qwen2.5:7b` (Q4), `llama3.1:8b` (Q4), `qwen2.5:14b` (Q4), and `llama3.1:8b-instruct-fp16`. Embeddings use Sentence-Transformers `all-MiniLM-L6-v2` (384-d) on CPU. The test suite comprises 161 cases: 120 intent, 16 ReAct, 20 safety, and 5 anomaly scenarios. Multi-run results (`qwen2.5:7b` and `llama3.1:8b`) report mean ± std over five independent runs.

3.1 Intent Classification

Table 1 compares 120 natural-language queries across 12 intent categories (search, read, write, delete, statistics, cluster monitoring, management, documentation, and edge cases).

The 3.8 B model falls below the usability threshold (< 60%), confirming that storage-domain intent classification requires

Table 1. Intent classification accuracy over 120 test queries. Multi-run entries (n=5) show mean ± std.

Model	Params	Intent Acc. (%)	Param Acc. (%)	Latency (ms)
phi3.5	3.8 B	55.8	72.5	6 324
llama3.1:8b	8 B	82.5 ± 0.8	91.0 ± 0.4	10 327
qwen2.5:7b	**7 B**	**85.5 ± 0.8**	**89.2**	**10 542**
qwen2.5:14b	14 B	85.0	88.3	13 575
llama3.1:8b-fp16	8 B	80.8	90.0	28 609

Table 2. ReAct vs. Simple mode on 16 diagnostic scenarios. Tool coverage is the fraction of expected tools invoked.

Model	Simple Succ.	ReAct Succ.	Tool Covg.	Avg Steps	ReAct Lat. (s)
phi3.5	0%	100%	3%	1.0	18.7
llama3.1:8b	81.2%	93.8%	71%	4.2	38.8
qwen2.5:7b	**75.0%**	**93.8%**	**58%**	**4.5**	**35.1**
qwen2.5:14b	81.2%	100%	60%	2.4	48.6
llama3.1:8b-fp16	81.2%	81.2%	67%	4.1	145.5

at least a 7 B model. Scaling from 7 B to 14 B yields *no accuracy gain* (85.5% vs. 85.0%) while increasing latency by 29%, establishing diminishing returns for on-node deployment. Perhaps counter-intuitively, the full-precision FP16 variant (80.8%) under-performs its Q4 counterpart (82.5%), suggesting that Ollama's Q4 quantisation preserves—or even regularises—the instruction-following ability required for structured function calls, while doubling inference time.

Error analysis. The remaining ∼15% errors concentrate in *management operations* (set_osd_out, create_pool, etc.) that the LLM has rarely seen during pre-training. Documentation-versus-explanation confusion (`search_docs` ↔ `explain_issue`) accounts for another cluster of misclassifications, reflecting inherent ambiguity in the queries.

3.2 ReAct Reasoning

We force each of 16 scenarios through both execution paths and compare (Table 2).

ReAct mode consistently outperforms simple mode on diagnostic tasks (93.8–100% vs. 0–81.2%), validating the dual-path design. The 14 B model solves tasks in fewer steps (2.4 vs. 4.5) but at higher wall-clock cost (48.6 s vs. 35.1 s). Mode routing achieves **100% accuracy** across all five models, confirming that keyword-based switching is sufficient—no learned classifier is needed.

3.3 Safety & Anomaly Detection

We test the action engine against 20 scenarios (8 LOW, 6 MEDIUM, 3 HIGH, 3 CRITICAL) covering read-only queries, reversible mutations, service-affecting operations, and cluster-wide flags. Risk classification achieves **100% accuracy** and confirmation gating reaches **95% accuracy** across *all five models*. This result is by construction: risk levels are determined by a dictionary lookup, not by LLM inference. The 5% confirmation gap stems from a policy edge case where MEDIUM-risk write operations are auto-approved under the default policy threshold.

Five synthetic cluster states—from perfectly healthy to compound failure (OSD down + near-full + degraded PGs)—are fed directly to the anomaly detector. All models yield identical results: **100% score accuracy** and **F1 = 1.0** (macro-precision and recall both 1.000). The detector correctly produces a score of 100 for the healthy cluster, 0 for the compound failure, and intermediate scores (30–65) for partial failures. These scores are model-independent by design.

Table 3. Latency profile (ms) over 10 operations × 5 iterations. CLI overhead is the ratio of agent to native ceph command latency.

Model	Mean	P50	P95	P99	CLI ×
phi3.5	5 041	4 000	9 012	17 944	6.7×
qwen2.5:7b	**6 072**	**5 759**	**8 726**	**8 819**	**8.3×**
llama3.1:8b	6 295	6 187	7 567	7 585	8.6×
qwen2.5:14b	8 880	8 893	10 099	10 145	12.6×
llama3.1:8b-fp16	14 790	15 057	17 972	18 279	21.7×

3.4 Latency

Table 3 profiles end-to-end latency for 10 representative operations, each repeated 5 times.

The 7 B Q4 models keep mean latency under 6.3 s—an 8–9× overhead over raw CLI that is acceptable for interactive operator sessions. The 14 B model exceeds the 10 s interactive threshold at P95, while FP16 inference is impractical at nearly 15 s mean. Latency is dominated by LLM inference; RADOS round-trips contribute < 700 ms on average.

4 Related Work

LLM agents for systems. Automated incident management has been explored in cloud settings [4], but these systems target microservice architectures and rely on cloud-hosted LLMs (GPT-4). Ceph-SRE runs entirely on-node with quantised open-weight models. Concurrent work on LLM-based SRE assistants [5] focuses on recommendation rather than autonomous execution.

ReAct and tool-augmented LLMs. Yao et al. [2] introduced the Thought-Action-Observation loop; Schick et al. [3] demonstrated self-taught tool use. We adapt ReAct specifically for storage operations with a domain-specific tool registry and safety interlock.

AIOps for storage. Prior AIOps work focuses on anomaly detection in telemetry [6], log analysis [7], and root-cause

analysis [8]. These approaches are complementary; Ceph-SRE integrates detection with autonomous remediation via runbooks.

Ceph management tooling. The Ceph Dashboard and `cephadm` provide GUI and orchestration interfaces but lack natural-language interaction or autonomous reasoning. Rook [9] automates Ceph lifecycle in Kubernetes but does not address interactive diagnostics.

5 Discussion

Model size sweet spot. Our results establish 7 B Q4 as the minimum viable model size for storage-domain agent tasks on a P100. The negligible accuracy gain from 7 B to 14 B (85.5% → 85.0%) suggests that domain-specific prompt engineering and structured function-call schemas already extract most of the available capability at 7 B. Future work will investigate whether fine-tuning on Ceph-specific corpora can close the remaining 15% accuracy gap.

Deterministic safety by design. The 100% safety and anomaly scores across all models are not coincidental—they are architectural. By implementing risk classification and threshold-based detection as dictionary lookups and arithmetic rather than LLM inference, we ensure that safety-critical components are unaffected by model hallucinations or degradation. This separation of concerns is essential for deploying LLM agents in production storage environments.

Latency–accuracy trade-off. The 8× CLI overhead is dominated by LLM inference and is acceptable for interactive management sessions (operators rarely need sub-second cluster diagnostics). For latency-sensitive monitoring, the anomaly detector runs independently of the LLM in < 1 ms.

Limitations. Our evaluation uses synthetic anomaly scenarios rather than live fault injection; a production deployment would benefit from chaos-engineering-style testing. The 120-query intent suite, while comprehensive, does not yet cover multi-tenant or RBAC-aware scenarios. The keyword-based router, though 100% accurate on our test set, may misroute novel phrasings in production.

6 Conclusion

We presented Ceph-SRE, a hybrid LLM agent for autonomous Ceph cluster management that runs on commodity GPU hardware. Our evaluation across five models and 161 test cases shows that: (1) a 7 B quantised model is sufficient for 85.5% intent accuracy at 6 s latency; (2) deterministic safety and anomaly components achieve 100% accuracy regardless of model quality; and (3) the dual-mode control loop achieves 100% routing accuracy with zero-cost keyword matching. These results demonstrate that on-node LLM agents are a practical replacement for manual operator scripts in storage cluster management.

References

[1] S. A. Weil, S. A. Brandt, E. L. Miller, D. D. E. Long, and C. Maltzahn. Ceph: A scalable, high-performance distributed file system. In *Proc. OSDI*, 2006.

[2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *Proc. ICLR*, 2023.

[3] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *Proc. NeurIPS*, 2023.

[4] X. Chen, et al. Automatic root cause analysis via large language models for cloud incidents. In *Proc. EuroSys*, 2024.

[5] T. Ahmed, et al. Recommending root-cause and mitigation steps for cloud incidents using large language models. In *Proc. ICSE*, 2023.

[6] N. Zhao, et al. An empirical investigation of practical log anomaly detection for online service systems. In *Proc. ESEC/FSE*, 2021.

[7] S. He, J. Zhu, P. He, and M. R. Lyu. A survey on automated log analysis for reliability engineering. *ACM Computing Surveys*, 54(6), 2021.

[8] J. Soldani and A. Brogi. Anomaly detection and failure root cause analysis in (micro)service systems: A survey. *ACM Computing Surveys*, 55(3), 2022.

[9] Rook Authors. Rook: Open-source, cloud-native storage for Kubernetes. https://rook.io/, 2024.